Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

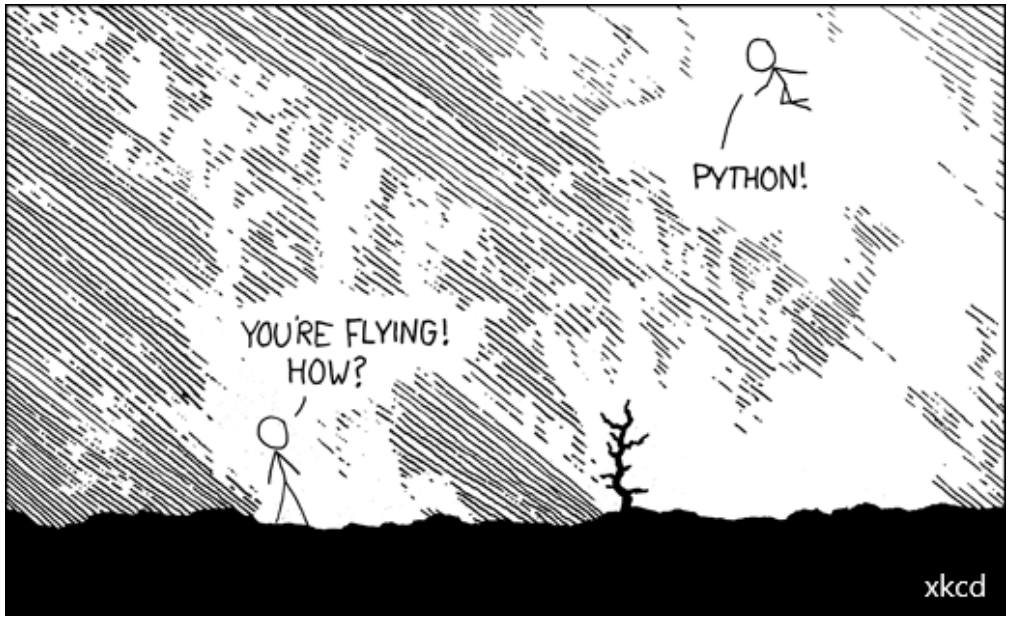
Introduction

Prof. Reza Skandari

Today

1. Welcome
2. **What this module is about**
3. Introductions
4. What we will do
5. Python exercises

What this module is about



- Computational problem-solving
- Understanding what computers can and cannot do efficiently
- Making a computer do these things

Algorithms!


Prisons turn to computer algorithms for deciding who to parole
By Jacob Kastrenakes on October 14, 2013 10:06 am
18
HEADLINES


We now have instant access to almost any song. Could our tastes be narrowing as a result?
DR. FEELGOOD
MEMBER BIEBER FAN CLUB
AC/DC
THE REZILLOS
U.K. SUBS
BLONDIE
Save


OF WALL STREET


The NSA revelations highlight the role sophisticated algorithms play in sifting through masses of data. But more surprising is their widespread use in our everyday lives. So should we be more wary of their power?


Sign in
News
Sport
Weather
World
Business
Politics
Tech
Science
Health
algorithms control the worl

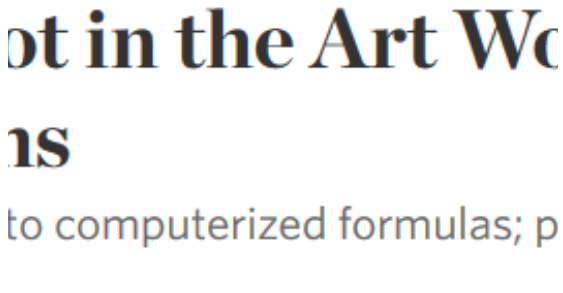

SCIENCE 08.06.15 1:24 PM
OGLE'S SEARCH ALGORITH
JLD STEAL THE PRESIDEN


f the DIY algo trac
urs beat the market —


ks May Have Beer
m
ng fish is a tough job... for humans. In this
mberg's Ashlee Vance visits a factory in Ic
anging the fish slicing game forever. (Sour


he Algorithm Is
ft, turn right: inside Orion, the
nny from delivery routes


cal Team: Algorithms
2015


ot in the Art Wo
ns
to computerized formulas; p


ns to Determ
, 2015 5:30 AM
82

What Will You Build?

Close your eyes for a second. It's the end of the programme — you can code.

What's the thing you've always wanted to build?

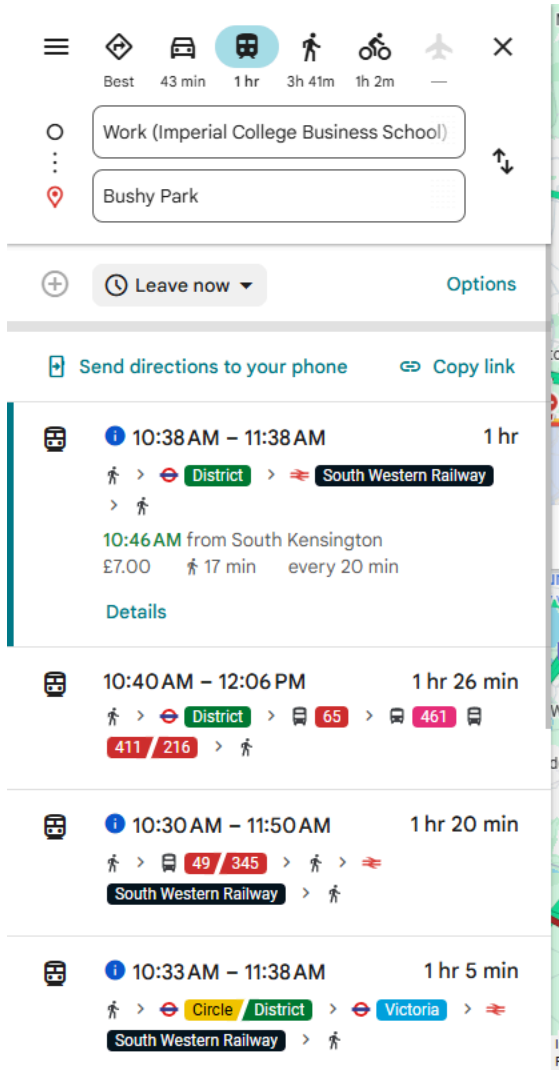
- The thing that fixes what frustrates you.
- The thing your industry, your hobby, or your community actually needs.
- Dream big — the algorithms you're about to learn are the toolkit. What's worth building with them?

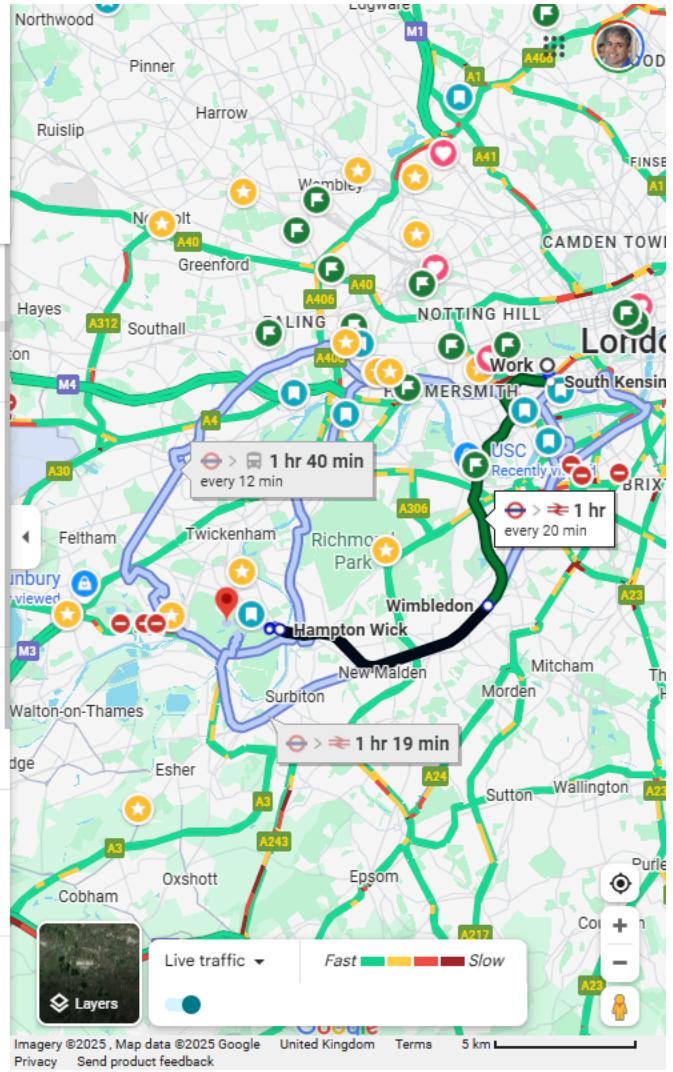
Talk with the people around you (2–3 students): what would it do, and who would it help?

We'll hear a few of your dreams before we dive in.

Going to Bushy Park from Imperial

1. You want to go to **Bushy Park** using public transport.
2. You use **Google Maps** to find the best route and method.
3. How does Google Maps do this? Think about the **data**, **structure**, and **process** behind it.





Solving computational problems

Data = digitised information

> **Data structures** describe ways to organise data
>
> **Algorithms** describe how we process data:
>
> - Step-by-step instructions
> - Take input data and produce output data
>
> We write algorithms into **programs** (eg in Python)

Computers interpret and execute programs

Programming languages

Python has brought computer programming to a vast new audience

And its inventor has just stepped down



Satoshi Kambayashi

Applications of Python

Video: What Can You Do with Python? (YouTube, 12 mins)

Data Science and AI

- **Data Science**: analysis, visualization, statistical modeling (pandas, matplotlib).
- **Scientific Computing**: simulations and numeric research (SciPy).
- **AI/Machine Learning**: predictive models and deep learning (scikit-learn, TensorFlow, PyTorch).

Web, Business, and Automation

- **Web Development**: back-end apps and APIs (Flask, Django).
- **Business Applications**: ERP systems, dashboards, automation tools.
- **Automation / Scripting**: file handling, data scraping, batch jobs.

Today

1. Welcome
2. What this module is about
3. **Introductions**
4. What we will do
5. Python exercises

Teaching team

Module leader

- Prof. Reza Skandari

Teaching assistants

- Alara Dirik
- Maksim Anisimov
- Furkan Pala
- Negar Maleki
- Yaqi Zhou

Quick poll

Let's get to know each other. Open menti.com on your phone — a few questions before we dive in.

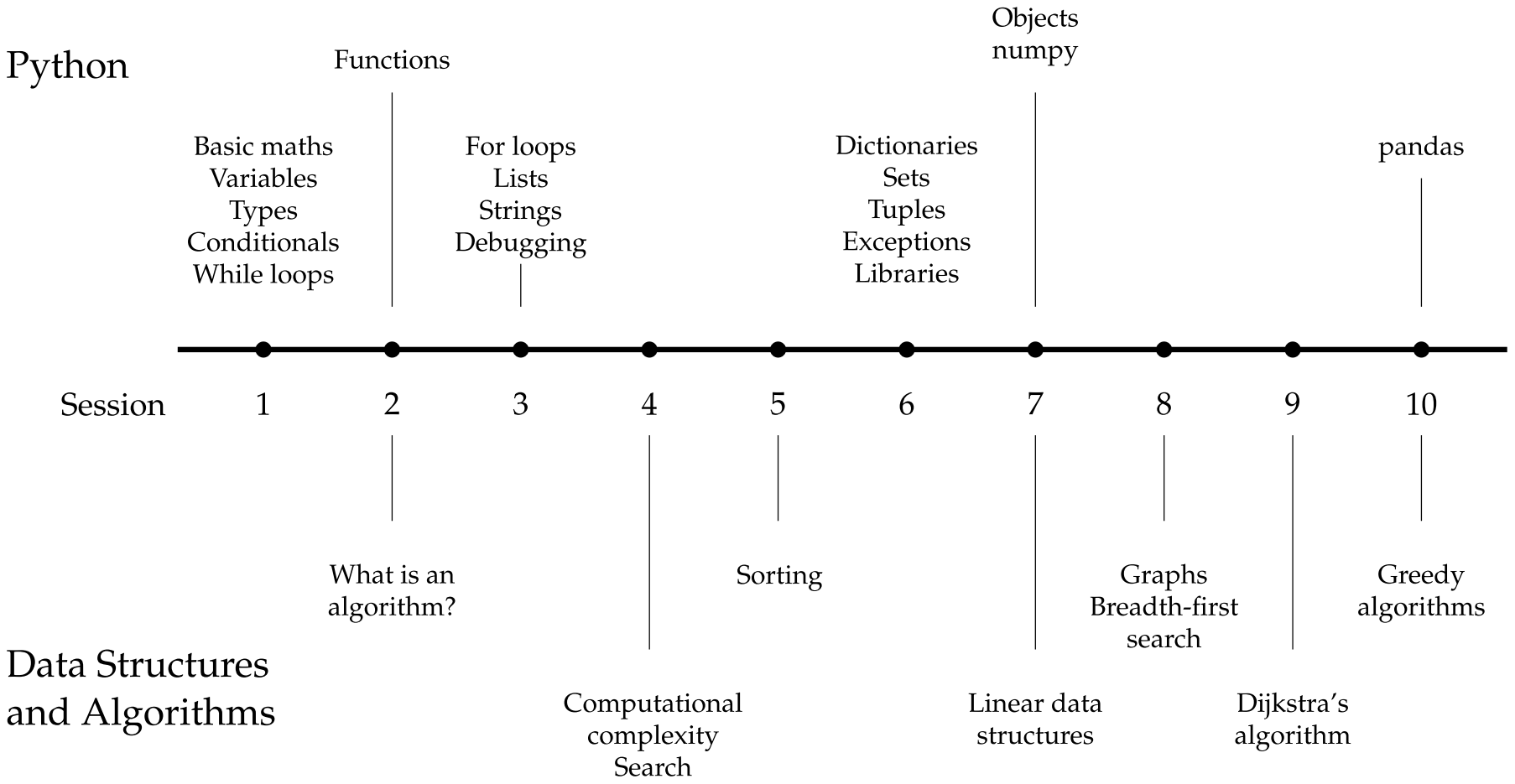
Today

1. Welcome
2. What this module is about
3. Introductions
4. **What we will do**
5. Python exercises

Philosophy

- Programmers are made, not born
- You will spend most of your time on this module working on exercises
- Mix of online materials and lectures: Lectures will focus on the most important ideas

What we will do



Using Generative AI in This Course

Our approach to GenAI (e.g., ChatGPT, Copilot)

Generative AI can help you write code faster and learn from examples — but only once you've built your own foundation.

First: build your own foundation

- Focus on understanding the *why* and *how* behind solutions, even when it's slower.
- Struggling through problems is how the skills stick.

Then: use AI as a productivity multiplier

- Speed up debugging, learn new syntax, explore alternatives.
- Always review and understand AI-generated code before using it.

Key takeaway: Master the fundamentals *first*, then let AI amplify your skills.

Generative AI: Good vs Bad Use in This Course

Good use (encouraged)

- Explaining a concept after you've tried it yourself.
- Suggesting test cases for code you've already written.
- Offering hints or alternatives to compare with your solution.
- Reviewing and refactoring code for clarity.

Bad use

- Copy-pasting solutions without understanding them.
- Solving graded exercises for you (plagiarism).
- Treating output as correct without testing it.
- Replacing your own thinking and debugging.

Key principle: Use AI as a *coach and collaborator*, not a shortcut past learning.

How to Prompt AI Effectively

1. Focus on learning, not just answers

- Ask for a step-by-step flow before code, e.g. "Explain the steps to check a palindrome."
- Try it yourself first; then ask: "Show me pseudocode."

2. Be clear and specific

- Task: "Sort a list of numbers."
- Constraints: "Avoid built-in sort functions."

3. Ask for explanations

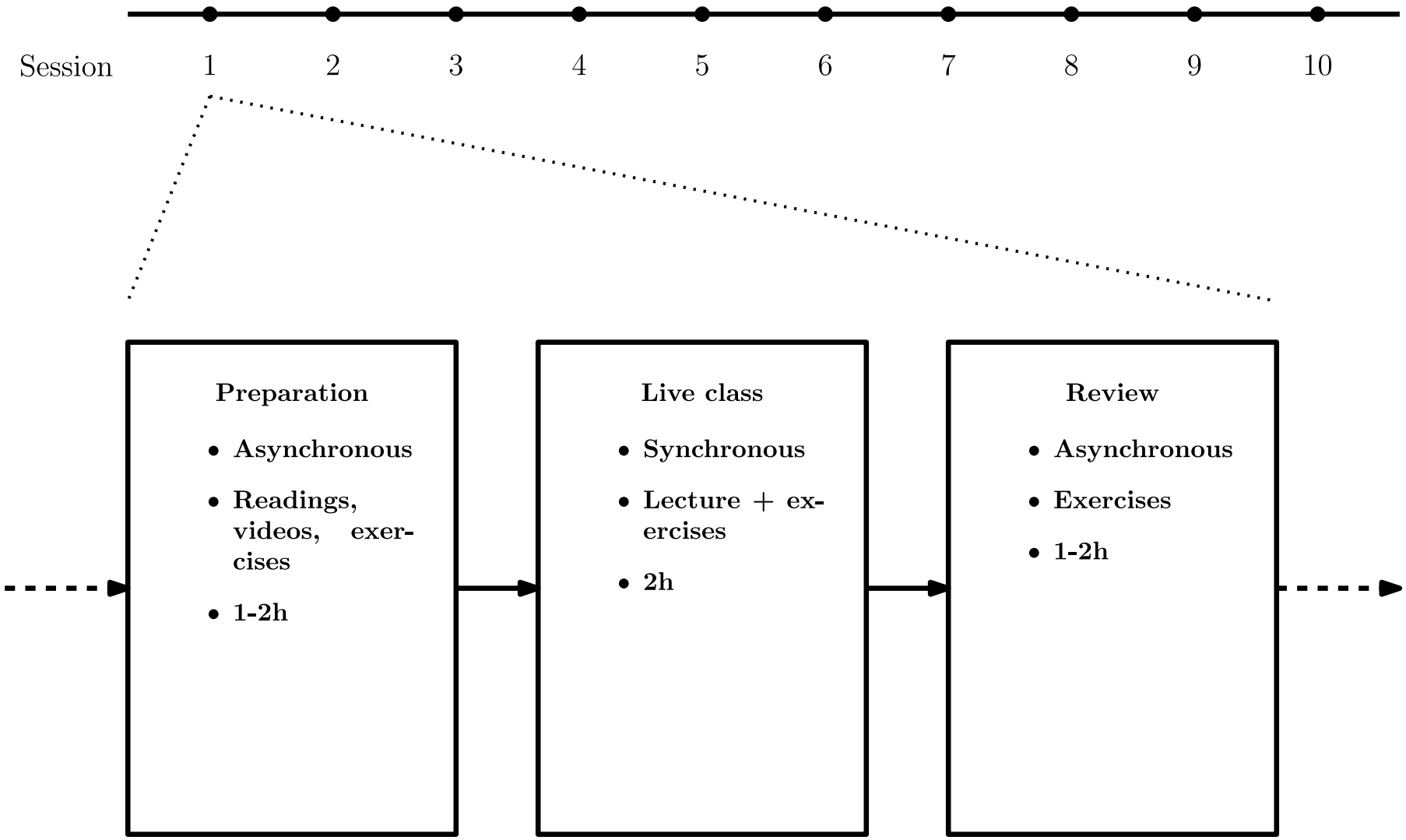
- "Why does this work, and what's its time complexity?"
- "Pros and cons of recursion vs. iteration here?"

4. Iterate and refine

- "Suggest an easier version for a beginner?"
- "Show me two approaches — which is simpler?"

Key takeaway: Use AI as a *tutor*, not a code generator.

Blended approach



Syllabus: Session 1 (now)

Session 1: Introduction to Python

Basic Python concepts: syntax, calculations, variables, assignment, and conditionals.

Preparation

1. Download the Getting Started Tutorial from Canvas: Python and GitHub Codespaces setup.
2. Complete the readings, and think about how you'd explain them to a classmate.

Lecture

Introductions, module goals, and getting started with Python.

Review

1. Work through the rest of session 1. This includes a number of Python exercises.

Assessment

Making the most of it

Preparation:

- Go through the preparation materials before the lecture.

Working on exercises:

- **Expect to debug:** Code usually won't work perfectly on the first go. Don't give up! Try to find the error and fix it. We'll discuss *debugging* in later sessions.
- **Collaborate wisely:** You may work together on the session exercises, but not on the individual homework assignments.
- **Ask for help:** If you get stuck, don't hesitate to reach out — getting unstuck quickly is key to learning.
- **Be selective:** You don't need to solve every exercise. Focus on understanding the core ideas.
- **Practice more:** Find additional challenges at edabit.com/challenges.

How to get help

1. **Ask during lectures**
   - We will work on exercises in the second half of each session.
   - Reza and TAs are around to help.
2. **Contact Reza**
   - Email on personal matters (e.g. grades).
   - With 400 students, I cannot guarantee quick email answers to Python questions.
   - Please mention your MSc programme and CID number in all emails.
   - Office hours by appointment only, on Zoom.

Today

1. Welcome
2. What this module is about
3. Introductions
4. What we will do
5. **Python exercises**

Python first steps

In this session, we will start working with Python:

- doing calculations
- storing the results in variables
- making decisions based on them

This requires learning the *syntax* and *semantics* of the Python language.

```
# "Will it rain today?"
rainfall = 2.3
if rainfall > 0.5:
    print('Bring an umbrella.')
else:
    print('No rain expected, enjoy your day!')
```

Code feedback using okpy

okpy:

- Code feedback software used in the course.
- Developed at UC Berkeley.
- Aimed at improving computer science education.

How we use it:

1. Testing your understanding as you work through the sessions.
2. Exercise grading.

What next

Short break, then:

1. Work through **Session 1**
    - Review the *Getting Started* PDF.
    - Work through the provided HTML file.
    - You may collaborate with neighbors or work individually.
    - Don't worry if you don't finish everything today.
    - Raise your hand if you need help.
2. Prepare for **Session 2** before the next lecture.